

ТЕСТОВОЕ ЗАДАНИЕ

Funnel Runtime

Конфигурируемые воронки, A/B, события, аналитика и rollback

Вакансия: Fullstack-разработчик

Срок: 48 часов с согласованного момента старта

Задача. Собрать мини-платформу для запуска, версионирования и анализа многошаговых веб-воронки. Все данные вымышленные; всё необходимое находится внутри одного репозитория.

Ограничение. Разрешены обычные open-source зависимости.

1. Динамическая воронка

Frontend не должен содержать захадркоженные экраны. Воронка загружается с backend в виде JSON-конфига. Вместе с заданием предоставляются два локальных конфига с вымышленным содержанием.

Нужно поддержать:

single-select;

multi-select;

числовой ввод;

информационный экран;

условные переходы между шагами;

валидацию;

экран результата и основной СТА.

В конфиге должно быть не менее шести экранов и хотя бы одно ветвление. Прогресс учитывает только доступные пользователю шаги. Back, refresh и повторное открытие страницы не должны терять состояние.

Визуальный редактор конфигов делать не нужно.

2. Версионирование и откаты

На backend хранятся версии конфигурации. Необходимо реализовать:

- публикацию новой версии без передеплоя;
- просмотр активной версии;
- откат на предыдущую версию;
- фиксацию версии за пользовательской сессией;
- продолжение старой сессии на старой версии после публикации новой;
- запуск новых сессий только на текущей активной версии.

Для управления достаточно простой внутренней страницы.

3. A/B-эксперимент

Внутри одной версии должны работать варианты A и B.

Вариант назначается на backend.

Назначение стабильно в рамках сессии и не меняется после refresh.

Для проверки разрешён override через query-параметр.

Вариант может менять тексты, порядок шагов и экран результата.

Все события содержат версию воронки и вариант эксперимента.

В README нужно кратко описать гипотезу эксперимента и основную метрику.

4. Событийная система

Нужно реализовать собственный endpoint приёма событий.

`session_started` — создана новая сессия;

`step_viewed` — пользователь увидел шаг;

`answer_submitted` — ответ отправлен;

`step_completed` — шаг завершён;

`back_clicked` — пользователь вернулся назад;

`result_viewed` — показан результат;

`cta_clicked` — нажат основной СТА.

4.1. Поля события

event_id;

session_id;

server timestamp и client timestamp;

версия воронки;

вариант эксперимента;

step ID;

UTM-метки;

дополнительные свойства конкретного события.

4.2. Инварианты

Повторная отправка одного event_id не создаёт дубль.

Endpoint принимает события пачкой.

Повторный запрос после timeout безопасен.

Некорректное событие не ломает обработку всей пачки.

Сырые ответы пользователя не попадают в аналитику без необходимости.

Для хранения используется SQLite или файловое хранилище внутри проекта.

5. Аналитический экран

Нужен внутренний dashboard, который считает показатели по уникальным сессиям, а не по количеству событий.

количество начавших воронку;

конверсия между шагами;

отвал на каждом шаге;

достижение результата;

CTR основного СТА;

сравнение вариантов А и В;

сравнение версий;

фильтрация по UTM campaign.

Расчёты должны корректно учитывать повторные просмотры, возвраты назад, дубли событий и события, пришедшие не по порядку.

6. Генератор тестового трафика

В репозитории должна быть команда, генерирующая минимум 100 синтетических сессий:

разные UTM-метки;

варианты А и В;

прохождение разных веток;

отвал на разных шагах;

повторные события;

повторная отправка одной пачки;

несколько событий, пришедших не по порядку.

После запуска генератора dashboard должен показывать осмысленные и проверяемые данные.

7. Технические условия

Fullstack TypeScript.

React или аналогичный frontend.

Backend на Node.js.

Один репозиторий.

SQLite или локальные файлы.

Публичный URL.

Никаких сторонних сервисов.

7.1. Минимальные автоматические тесты

закрепление версии за сессией;

стабильность A/B-варианта;

дедупликация событий;

публикация и откат версии;

расчёт основных аналитических показателей.

8. Вторая итерация

После первой рабочей версии кандидат получает новый локальный JSON-конфиг со следующими изменениями:

добавляется условная ветка;

один экран удаляется для варианта B;

появляется новое событие;

старые активные сессии должны продолжить работу без ошибок.

Нужно опубликовать новую версию, проверить её, затем выполнить откат — без ручного изменения схемы базы и без потери существующей аналитики. Вторая итерация входит в общий срок 48 часов.

9. Что нужно прислать

- ссылку на работающий публичный URL;
- ссылку на репозиторий;
- README с инструкцией локального запуска;
- описание модели данных, event schema и правил агрегации;
- описание A/B-гипотезы и основной метрики;
- краткий таймлайн первой и второй итерации;
- известные ограничения и принятые допущения.

10. Использование AI-агентов

Использование Claude Code, Cursor, Codex или аналогичных инструментов ожидается.

Важно не формальное количество агентов, а способность управлять ими, проверять результат и полностью отвечать за итоговую реализацию.

11. Как оцениваем

Критерий	Вес	Что проверяем
Работоспособность и скорость	25%	Первая версия, вторая итерация и отсутствие регрессий.
Состояние, версии и rollback	20%	Закрепление версии, публикация, совместимость старых сессий и откат.
События и аналитика	20%	Идемпотентность, batching, дедупликация и корректные агрегаты.
Продуктовые решения	20%	Логика воронки, A/B-гипотеза, UX ветвлений и интерпретация метрик.

Критерий	Вес	Что проверяем
Агентский процесс	15%	Декомпозиция, параллельная работа, ревью и понимание сгенерированного кода.